Лабораторная работа 2

Архитектурные inductive biases в компьютерном зрении

CNN vs Vision Transformer: sample efficiency, augmentation, transfer и distribution shift

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Свёрточная сеть (CNN) и Vision Transformer (ViT) технически решают одну задачу: определяют, что находится на изображении. Различие — в предположениях о структуре изображения заложенных в архитектуру до обучения. Такие предположения называют *индуктивными смещениями* (inductive bias).

Свёртка смотрит на маленькое окно и применяет одни и те же веса во всём изображении. У слоя 3×3 с 64 каналами на входе и выходе ядро содержит $64 \cdot 64 \cdot 9 = 36\,864$ веса, без смещений. Полносвязный слой между картами $64 \times 32 \times 32$ имел бы $(64 \cdot 32 \cdot 32)^2 = 4{,}29$ млрд весов. Локальное окно без общих весов оставляет 37,7 млн весов: меньше примерно в 114 раз. Общее ядро уменьшает это число ещё в $32 \cdot 32 = 1024$ раза.

ViT предполагает меньше. Он режет изображение на квадратные фрагменты — патчи. Уже в первом блоке любой патч может влиять на любой другой. Представление о связи соседних патчей предстоит выучить из данных. На малых данных можно ждать преимущества CNN, а с ростом данных — сокращения разрыва.

В статье, где предложен ViT, модель близкого размера уступала ResNet на ImageNet, около 1,3 млн изображений [13]. Крупный ViT, обученный на наборах около 14 млн и 300 млн изображений, ResNet обгонял. Вопрос «CNN или ViT лучше?» не имеет ответа без объёма данных и условий обучения. Сравнивать архитектуры имеет смысл при сопоставимых параметрах и вычислениях. Иначе «A лучше B» может означать «A просто больше».

Постановка экспериментов также основана на проекте из ЛР0. У каждого запуска есть `EXP_ID`, `RUN_ID`, паспорт и итоговая конфигурация. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель — выяснить, когда встроенные предположения CNN дают ей преимущество перед ViT того же размера. Итог — **карта относительного преимущества**. Она показывает,

как разность точности ViT и CNN зависит от объёма данных и от искажений теста. Это не таблица «какая модель точнее всех».

Когда индуктивные смещения архитектуры важнее числа её параметров и как это зависит от объёма данных, аугментации, предобучения и сдвига распределения?

3 Предполагаемые предварительные знания

Необходимые пререквизиты для выполнения работы. Если какой-то пункт незнаком, можно обратиться по указанному источнику.

- Свёртка, пулинг, свёрточные сети и ResNet — [1, разд. 5.2, 5.4], [2, разд. 9.1–9.3], [11]. Остаточный блок и нормализация — ЛР1.
- Механизм внимания и Transformer — [1, разд. 8.1], [12].
- Vision Transformer — документация Hugging Face [8] и курс по компьютерному зрению [9].
- Аугментация, то есть пополнение набора данных, — [2, разд. 7.4]. Преобразования torchvision — [4].
- Готовые модели torchvision и timm — [3, 10]. Хуки PyTorch — [6].
- Проект для экспериментов из ЛР0.

4 Результаты обучения

После выполнения работы вы сможете:

1. объяснять индуктивное смещение как встроенное предположение о данных и называть смещения свёртки, которых нет у ViT;
2. считать параметры и вычисления свёрточного слоя и блока ViT;
3. сравнивать модели при равных параметрах или равных вычислениях;
4. строить кривые точности по объёму данных и находить долю, где меняется лидер;
5. проверять устойчивость к искажениям тестовых изображений;
6. делать вывод по парным разностям на трёх seeds и указывать границу этого вывода.

5 Модели и обозначения

5.1 Свёрточная сеть

Свёрточный слой скользит по входу окном — *ядром* — и в каждой позиции считает взвешенную сумму. Выход канала c' в позиции (i, j):

$$(w * h)_{c',i,j} = \sum_{c=1}^{C} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} w_{c',c,u,v}\, h_{c,\ \sigma i+u-p,\ \sigma j+v-p} + b_{c'}. \tag{2.1}$$

Здесь h — вход с C каналами, высотой H и шириной W. Ядро w имеет C' выходных каналов и размер $k \times k$, b — смещения. Шаг σ задаёт, на сколько пикселей сдвигается

окно. Краевое дополнение p дописывает нули за границей входа. Индексы u, v бегут по окну, c — по входным каналам.

В формулу встроены два предположения. **Локальность:** выход зависит только от окна $k \times k$. **Общие веса:** ядро одно на все позиции, поэтому число параметров $C'Ck^2 + C'$ не зависит от H и W. Пример из введения — это как раз эти два сокращения. С глубиной растёт *рецептивное поле*, область входа, от которой зависит выход. После двух свёрток 3×3 с шагом 1 оно равно 5×5, после трёх — 7×7.

Третье свойство — *эквивариантность к сдвигу*. Сдвиг входа сдвигает выход так же. Для сдвига T_t на целочисленный вектор t, при $\sigma = 1$ и вдали от краёв,

$$w * (T_t h) = T_t (w * h). \tag{2.2}$$

При $\sigma > 1$ сдвиг входа на вектор, кратный σ, сдвигает выход на этот вектор, делённый на σ. Другие сдвиги меняют выход сложнее. Нулевое дополнение и пулинг тоже нарушают (2.2). Поэтому выход реальной CNN меняется даже при сдвиге на один пиксель [15].

CNN в работе — ResNet-18 [11]. Остаточный блок устроен как в ЛР1, но ветвь — две свёртки 3×3 с нормализацией BatchNorm. Для изображений 32×32 первая свёртка заменена на 3×3 с шагом 1, а максимальный пулинг убран. Из четырёх стадий три уменьшают сторону карты вдвое, поэтому общий шаг сети равен 8. В конце карта усредняется по позициям и идёт в линейный классификатор.

Подробнее: [1, разд. 5.2, 5.4]; [2, разд. 9.1–9.4]; готовые модели — [3].

5.2 Vision Transformer

ViT [13] режет изображение на патчи $P \times P$. Каждый патч вытягивается в вектор и линейно отображается в *токен* длины D:

$$N = \frac{H}{P} \cdot \frac{W}{P}, \qquad z_i^0 = W_{\text{in}} x_i + b_{\text{in}} + e_i, \qquad i = 1, \ldots, N. \tag{2.3}$$

Здесь N — число патчей, x_i — патч i длины P^2C. Матрица W_{in} имеет размер $D \times P^2C$, вектор b_{in} — смещение. Вектор e_i — обучаемое *позиционное вложение* (positional embedding) позиции i. При $P = 4$ изображение CIFAR-10 даёт $N = 64$ патча. В начало последовательности добавляется обучаемый *классовый токен*. Тогда длина последовательности $N' = N + 1$. Предсказание делается по нему. Выбор записывается в конфигурацию.

Последовательность проходит через L блоков Transformer [12]. Нормализация стоит до ветви, а не после неё:

$$z' = z + \mathrm{MHSA}\big(\mathrm{LN}(z)\big), \qquad z^+ = z' + \mathrm{MLP}\big(\mathrm{LN}(z')\big). \tag{2.4}$$

Матрицы z, z' и z^+ имеют размер $N' \times D$: вход блока, выход внимания и выход блока. LN — LayerNorm по признакам одного токена, как в ЛР1. MLP — два линейных слоя с активацией GELU. Скрытый слой имеет ширину $4D$ и применяется к каждому токену отдельно.

Токены обмениваются информацией только во *внимании* (self-attention). Каждый токен строит запрос, ключ и значение. Выход токена — сумма значений всех токенов. Вес тем больше, чем ближе запрос к ключу:

$$\begin{gathered} \mathrm{head}_a = \mathrm{softmax}\!\left(\frac{Q_a K_a^\top}{\sqrt{d_k}}\right) V_a, \qquad Q_a = zW_a^Q,\ K_a = zW_a^K,\ V_a = zW_a^V, \\ \mathrm{MHSA}(z) = \big[\mathrm{head}_1, \ldots, \mathrm{head}_{n_\mathrm{h}}\big]\, W^O. \end{gathered} \tag{2.5}$$

Здесь n_{h} — число голов, a — номер головы, $d_k = D/n_{\text{h}}$ — размер головы. Матрицы W_a^Q, W_a^K и W_a^V имеют размер $D \times d_k$. Функция softmax превращает каждую строку в неотрицательные веса с суммой 1. Скобки $[\cdot]$ склеивают головы, W^O — выходная матрица $D \times D$.

Из (2.4) и (2.5) следуют три свойства. В первом блоке любой токен видит любой другой: локальности нет. Без векторов e_i блок эквивариантен к перестановке патчей. Предсказание по классовому токену или по среднему при этом не меняется. Внимание с девятью головами и относительным позиционным кодированием может повторить свёртку 3×3 [14]. ViT не обязан этому научиться. О положении патчей модель узнаёт из e_i, а порядок пикселей внутри патча хранится в x_i.

Подробнее: [1, разд. 8.1]; [8, 9]; реализация внимания — [5].

5.3 Число параметров и вычисления

Ресурсы модели m — число обучаемых параметров $|\theta_m|$ и число умножений с накоплением $\mathcal{C}_m$ на один прямой проход. Одна такая операция — умножение и сложение. Для свёртки (2.1) с выходом $H' \times W'$ и для блока (2.4)

$$\mathcal{C}_{\text{conv}} = H'W'C'Ck^2, \tag{2.6}$$

$$\mathcal{C}_{\text{block}} \approx 4N'D^2 + 2N'^2D + 8N'D^2. \tag{2.7}$$

Первое слагаемое (2.7) — проекции Q, K, V и W^O. Второе — произведения $Q_a K_a^\top$ и весов внимания на V_a. Третье — два слоя MLP. Нормализация, softmax и GELU в сумму не входят. Второе слагаемое квадратично по N'. Если уменьшить P вдвое, оно вырастет примерно в 16 раз.

Пример посчитан для классового токена, то есть $N' = 65$. ResNet-18 для 32×32 имеет 11,17 млн параметров и $5{,}55 \cdot 10^8$ операций. ViT с $P = 4$, $D = 384$, $n_{\text{h}} = 6$ и $L = 6$ имеет 10,70 млн параметров и $7{,}11 \cdot 10^8$ операций. По параметрам разница около 4%. По вычислениям ViT дороже на 28%. Поэтому режим сравнения записывают явно.

- **Равные параметры:** $|\theta|$ двух моделей различаются не более чем на 20%.
- **Равные вычисления:** то же условие для $\mathcal{C}_m$.
- **Стандартные конфигурации:** модели берутся как есть, оба числа указываются.

Хотя бы один главный вывод делается в первом или втором режиме. Задержка, пропускная способность и память дополняют $\mathcal{C}_m$, но зависят от кода и оборудования.

Подробнее: подсчёт через хуки — [6]; измерение времени — [7].

5.4 Как сравниваются архитектуры

Главная величина работы — *относительное преимущество* ViT над CNN в режиме ξ:

$$\Delta(\xi) = M_{\text{ViT}}(\xi) - M_{\text{CNN}}(\xi). \tag{2.8}$$

Режим ξ — это доля обучающих данных, искажение теста и его уровень. M — точность (accuracy). При $\Delta > 0$ точнее ViT. Карта относительного преимущества — значения $\Delta(\xi)$ по режимам работы.

Объём данных. Модель учится на доле r части `train`, то есть на $n_r = \lfloor r\, n \rfloor$ объектах, где $n = 43\,000$. Кривая $r \mapsto M(r)$ — это *эффективность по данным* (sample efficiency). Порог смены лидера

$$r^* = \min\{\, r \in \mathcal{R} : \Delta(r) \geq 0 \,\}, \tag{2.9}$$

где $\mathcal{R}$ — доли из профиля. Если $\Delta(r) < 0$ при всех этих долях, порога в диапазоне нет. Это допустимый результат.

Длительность задаётся числом шагов оптимизатора S при размере batch B, а не числом эпох. Длительность в E эпох на доле r — это

$$S_E(r) = E \left\lceil \frac{n_r}{B} \right\rceil \tag{2.10}$$

шагов. На малой доле обновлений во много раз меньше. Кривая $M(r)$ смешала бы тогда объём данных и число обновлений. Все запуски получают одно и то же S. Число эпох $E_r \approx SB/n_r$ записывается. При $S = 10^4$ и $B = 128$ это около 30 эпох при $r = 1$ и около 595 эпох при $r = 0{,}05$.

Сдвиг распределения. Тестовые изображения искажаются преобразованием c уровня $s = 1, \ldots, s_{\max}$. Относительное падение точности и средняя точность по уровням:

$$\delta_c(s) = \frac{M_{\text{clean}} - M_c(s)}{M_{\text{clean}} + \varepsilon}, \qquad \overline{M}_c = \frac{1}{s_{\max}} \sum_{s=1}^{s_{\max}} M_c(s). \tag{2.11}$$

Здесь M_{clean} — точность на чистом `test`, $M_c(s)$ — на `test`, искажённом преобразованием c уровня s, $\varepsilon = 10^{-12}$. *Смена лидера* — случай, когда одна модель точнее на чистом `test`, а другая точнее по $\overline{M}_c$ хотя бы для одного искажения.

Подробнее: [17, 18].

5.5 Сводка обозначений

Обозначение	Смысл
h, C, H, W; w, b, k	вход слоя и его размеры; ядро, смещения, размер ядра
σ, p, C'; T_t	шаг, дополнение, число выходных каналов; сдвиг
P, N, N'; x_i, e_i, z	размер патча, число патчей и токенов; патч, вложение, матрица токенов
D, L, n_{h}, d_k	ширина ViT, число блоков, число и размер голов
$\lvert\theta_m\rvert$, $\mathcal{C}_m$, M, Δ	параметры, операции, точность, преимущество ViT
r, n, n_r, $\mathcal{R}$, r^*	доля, объёмы, набор долей, порог смены лидера
S, B, E; c, s, δ_c, $\overline{M}_c$	шаги, batch, эпохи; искажение, уровень, падение, средняя точность

6 Данные и оборудование

Основной набор — CIFAR-10 [16]: цветные изображения 32×32, 10 классов. Разбиение из ЛР0: `train` — 43 000, `val` — 5 000, `cal` — 2 000, `test` — 10 000. По `val` выбирают скорость обучения и режим E3. Итоговые числа считают на `test`. Часть `cal` не читается до ЛР8. Правила разбиения и кадры лиц для дополнительного задания 2 — в документе «Протокол данных курса».

Оборудование. Желательна видеокарта: своя, Google Colab или Kaggle. Без неё выполните уменьшенный вариант из задания 1, с $S = 3 \cdot 10^3$ шагов. Протокол тот же. Вариант укажите в отчёте.

7 Индивидуальный профиль

Профиль ЛР2 считается функцией `lab_pick` из ЛР1 с номером работы 2. Добавьте в генератор профиля три поля:

```
profile["lab2_fractions"] = lab_pick(sid, 2, 0, 2, [
    [0.05, 0.10, 0.25, 0.50, 1.00], [0.05, 0.15, 0.30, 0.60, 1.00]])
profile["lab2_corruptions"] = lab_pick(sid, 2, 2, 4, [
    ["gaussian_noise", "blur", "patch_shuffle"],
    ["contrast", "occlusion", "translation"],
    ["gaussian_noise", "occlusion", "translation"],
    ["blur", "contrast", "patch_shuffle"]])
profile["lab2_strong_aug"] = lab_pick(sid, 2, 4, 6, [
    "randaugment+cutmix", "trivialaugment+mixup"])
```

Здесь `sid` — номер зачётки, `profile` — словарь профиля из ЛР0. Профиль задаёт доли r для эксперимента E1, три искажения для E2 и сильную политику аугментации A_2 из задания 2. В каждом наборе одно искажение меняет расположение пикселей, а два меняют их значения.

8 Задания

8.1 Задание 1. Единый интерфейс моделей и профилировщик

Что сделать.

1. Возьмите готовые модели. Писать их с нуля не нужно. Эталонная пара собирается так:

   ```
   cnn = resnet18(num_classes=10)
   cnn.conv1 = nn.Conv2d(3, 64, 3, stride=1, padding=1, bias=False)
   cnn.maxpool = nn.Identity()
   vit = VisionTransformer(
       image_size=32, patch_size=4, num_layers=6, num_heads=6,
       hidden_dim=384, mlp_dim=1536, num_classes=10)
   ```

   Класс `VisionTransformer` — из torchvision [3]. Модель timm с теми же размерами допустима, если режим сравнения соблюдён.

2. Уменьшенный вариант для CPU. ResNet-8:main 3×3, три остаточных блока шириной 64, 128 и 256 каналов с шагами 1, 2 и 2, затем классификатор. Около 1,23 млн параметров. ViT-tiny: те же аргументы, но `hidden_dim=192`, `num_heads=3`, `num_layers=3`, `mlp_dim=768`, около 1,36 млн параметров. Ширина и число голов — как у DeiT-Ti, глубина меньше [21].

3. Оберните обе модели в один интерфейс:

   ```
   logits = model(x)                    # (B, 10)
   feats = model.forward_features(x)    # (B, d)
   ```

```
layers = model.forward_features(x, return_intermediate=True)
```

Промежуточные представления нужны в дополнительном задании 3.

4. Профилировщик пишет в каталог запуска: число параметров, отдельно обучаемых; $\mathcal{C}_m$; пиковую память видеокарты; задержку при $B = 1$; пропускную способность при $B = 128$; время обучения.

5. Операции свёрток и линейных слоёв считайте хуками [6]. У `nn.MultiheadAttention` проекции и произведения внимания хук на `nn.Linear` не видит: модуль умножает на веса напрямую [5]. Для ViT из примера это 35% суммы. Добавьте недостающую часть (2.7). Одни произведения $QK^\top$ и AV дают ошибку больше 2,5%.

6. Время — медиана 30 замеров после 10 прогревочных прогонов [7]. Перед замером на видеокарте вызовите `torch.cuda.synchronize()`. Режим сравнения запишите в конфигурацию: `param_matched`, `compute_matched` или `canonical`.

Что должно получиться. Обе модели вызываются одинаково. Паспорт ресурсов любой модели строится одной командой.

Как проверить себя. Формы выходов совпадают (тест 2). Число операций одиночной свёртки совпадает с (2.6) (тест 3). У эталонной пары — числа из раздела 5.3.

8.2 Задание 2. Режимы данных и аугментации

Что сделать.

1. Выберите подмножество доли r: n_r объектов части `train`, поровну из каждого класса. Подмножества вложены:

$$\mathcal{D}_{r_1} \subset \mathcal{D}_{r_2} \subset \cdots \subset \mathcal{D}_{r_5} = \mathcal{D}_{\text{train}}, \qquad r_1 < \cdots < r_5 = 1. \tag{2.12}$$

С ростом r объекты только добавляются. Разница точек кривой отражает именно рост данных. $\mathcal{D}_r$ — подмножество доли r, а $r_1, \ldots, r_5$ — доли профиля.

2. Перемешайте индексы каждого класса один раз, генератором с `base_seed` профиля ЛР0. Берите первые $\lfloor r \cdot 4300 \rfloor$ индексов класса. Индексы сохраните в паспорт: их отпечаток входит в `EXP_ID`.

3. Остановка — ровно после S шагов при любом r. Число эпох E_r запишите в сводку.

4. Реестр политик аугментации [4]:

   - A_0 — без аугментации: перевод в тензор и нормализация по средним и разбросам каналов, посчитанным на `train`;
   - A_1 — слабая: случайный фрагмент 32×32 после рамки в 4 пикселя и отражение по горизонтали;
   - A_2 — сильная, из профиля: RandAugment [22] с CutMix [25] либо TrivialAugment [23] с MixUp [24].

   Имя политики сохраняется в итоговой конфигурации.

Что должно получиться. Режим «доля r, политика A_k» задаётся конфигурацией, без правки кода.

Как проверить себя. Подмножества воспроизводимы, вложены и сбалансированы (тест 4). Политика A_0 детерминирована (тест 5). При $r = 0{,}05$ в подмножестве 2150 изображений, по 215 на класс.

8.3 Задание 3. Набор искажений

Что сделать.

1. Исказите тестовую часть преобразованием c уровня s [17]:

$$\mathcal{D}_{\text{test}}^{(c,s)} = \big\{(T_{c,s}(x),\, y) : (x, y) \in \mathcal{D}_{\text{test}}\big\}. \tag{2.13}$$

   Здесь x — изображение, y — метка, $T_{c,s}$ — преобразование. При $s = 0$ оно тождественно. Метка не меняется.

2. Реализуйте шесть искажений. Уровень s растёт слева направо.

Искажение	Что меняется	s=1	s=2	s=3
`gaussian_noise`	стандартное отклонение шума	0,04	0,08	0,12
`blur`	стандартное отклонение ядра, пикс.	0,5	1,0	1,5
`contrast`	доля отклонения от средней яркости	0,6	0,4	0,2
`occlusion`	сторона серого квадрата, пикс.	6	10	14
`translation`	сдвиг вдоль одной оси, пикс.	2	4	8
`patch_shuffle`	сторона блока, пикс.	16	8	4

3. Яркости искажайте в $[0, 1]$ до нормализации и обрезайте до $[0, 1]$. Сторона ядра размытия — $2\lceil 3\sigma \rceil + 1$. Контраст — это средняя яркость плюс указанная доля отклонения от неё. Сдвиг дописывает нули, ось и знак задаёт seed. Решётка `patch_shuffle` начинается в левом верхнем углу. Меньший блок сильнее ломает картинку, поэтому $s = 3$ сильнее $s = 1$. Блок в 4 пикселя совпадает с патчем ViT.

4. Шум, положение квадрата, направление сдвига и перестановку порождайте генератором с фиксированным seed. Тогда обе модели видят одни и те же искажённые изображения.

Что должно получиться. Для обученной модели одна команда строит таблицу $M_c(s)$ по всем искажениям и уровням. Имя, уровень, параметры и seed записаны в паспорт.

Как проверить себя. При $s = 0$ точность совпадает с точностью на чистом `test`. Метки не меняются (тест 6).

Что добавляется в общий проект. Добавьте файлы `src/models/vision/resnet.py`, `vit.py`, `src/profiling/model_profiler.py`, `src/data/regimes.py`, `augmentations.py`, `src/diagnostics/corruptions.py` и `src/analysis/learning_curves.py`. Сохраните контрольные точки при $r = 1$: они и набор искажений нужны в ЛР8.

9 Эксперименты

Перед каждым экспериментом запишите гипотезу: что вы ожидаете и почему, со ссылкой на формулу раздела 5. E1 и E2 — поисковые, достаточно одного seed $j = 1$. Главный вывод проверяется в E3 на трёх seeds. Расхождение с гипотезой допустимо, если объяснено, какое предположение не выполнилось.

Общий протокол одинаков для обеих моделей.

- Оптимизатор AdamW. Затухание весов рекомендуется 0,05, прогрев — 5% шагов, далее косинусное расписание. Эти три величины общие и не подбираются.
- $S = 10^4$ шагов при $B = 128$. В уменьшенном варианте $S = 3 \cdot 10^3$. Политика аугментации — A_1.
- Скорость обучения выбирается по `val` при $r = 1$ из сетки $\{3 \cdot 10^{-4},\ 10^{-3},\ 3 \cdot 10^{-3}\}$, отдельно для каждой модели, и дальше не меняется.

Другого подбора нет. Разница в настройке тоже меняет точность [19, 20].

9.1 Эксперимент E1. Эффективность по объёму данных

Вопрос. Как меняется преимущество одной архитектуры с ростом объёма обучающих данных, если модели сопоставимы по ресурсам?

Что сделать.

1. Снимите паспорта ресурсов из задания 1. Сравните измеренные $\mathcal{C}_m$ с суммой формул (2.6) и (2.7) по всем слоям. Укажите режим сравнения.
2. Обучите обе модели на каждой доле r из профиля. Точность считайте на `val` и на `test`. Режим для E3 выбирайте по `val`. На графики выносите `test`.
3. Постройте графики $M_{\text{CNN}}(r)$ и $M_{\text{ViT}}(r)$ с логарифмической осью r и график $\Delta(r)$ (2.8). Найдите r^* (2.9). Отдельно отметьте точки «время обучения — точность» обеих моделей при $r = 1$.

Порог приёмки.

- $|\theta_{\text{ViT}}|$ отличается от $|\theta_{\text{CNN}}|$ не более чем на 20%. В режиме равных вычислений то же требуется от $\mathcal{C}_m$.
- Измеренные $\mathcal{C}_m$ отличаются от расчёта не более чем на 1%.
- Каждый запуск сделал ровно S шагов.
- Для всех долей получены обе точности и $\Delta(r)$. Указан r^* или его отсутствие.

Граница применимости. Если r^* найден, дальше по диапазону ViT не хуже CNN при этом протоколе. Если нет, лидер в диапазоне не сменился. Продолжить кривую за $r = 1$ по этим точкам нельзя: новых изображений CIFAR-10 в части `train` нет. Другой рецепт обучения может сдвинуть r^* [19, 20].

9.2 Эксперимент E2. Сдвиг распределения

Вопрос. Сохраняется ли порядок архитектур на искажённых тестовых изображениях и какие искажения сильнее бьют по каждой модели?

Что сделать. Возьмите модели E1, обученные при $r = 1$. Новое обучение не нужно. Для трёх искажений профиля и уровней $s = 0, \ldots, 3$ вычислите $M_c(s)$, $\delta_c(s)$ и $\overline{M}_c$ (2.11). Для каждого искажения постройте график $s \mapsto M_c(s)$ обеих моделей. Сведите числа в таблицу и отметьте, есть ли смена лидера [18].

Порог приёмки.

- При $s = 0$ точность совпадает с M_{clean} точно.
- Для трёх искажений и трёх уровней $s \geq 1$ получены $M_c(s)$, $\delta_c(s)$ и $\overline{M}_c$ обеих моделей.
- Смена лидера засчитывается, если разности точностей на чистом `test` и по $\overline{M}_c$ имеют разные знаки и обе по модулю не меньше 1 процентного пункта. Это удвоенная наибольшая стандартная ошибка доли на 10 000 изображениях: $2\sqrt{0{,}25/10\,000} = 0{,}01$.

Граница применимости. Для искажения, которое двигает пиксели, объясните результат формулами (2.2) и (2.3). Вывод относится только к искажениям, сохраняющим метку. На естественные сдвиги данных он без проверки не переносится. Смена лидера на одном seed — поисковый результат. Если она выбрана главным выводом, её проверяет E3.

9.3 Эксперимент E3. Основное сравнение

Вопрос. Выдерживает ли главный вывод работы смену seed?

Что сделать. По результатам E1 и E2 выберите режим ξ^* главного вывода. Например, обе модели на наименьшей доле профиля или на доле, ближайшей к r^*. До запусков запишите гипотезу в файл `hypothesis.md` и зафиксируйте её в Git.

1. Что сравнивается: две архитектуры в режиме ξ^*.
2. Что зафиксировано: модели, подмножество $\mathcal{D}_r$, число шагов S, скорости обучения, политика A_1, seeds.
3. Какой знак и какую величину средней разности вы ожидаете и почему. Ссылка — на формулу или на результат E1–E2.
4. Какой результат опровергнет гипотезу.

Обучите обе модели с тремя общими seeds $j = 1, 2, 3$. Запуски E1 с $j = 1$ входят в их число. Подмножество $\mathcal{D}_r$ одно для всех seeds. Парные разности:

$$\Delta_j = M_{\text{ViT}}(\xi^*, j) - M_{\text{CNN}}(\xi^*, j), \quad \bar{\Delta} = \frac{1}{3}\sum_{j=1}^{3} \Delta_j, \quad s_\Delta = \sqrt{\frac{1}{2}\sum_{j=1}^{3}(\Delta_j - \bar{\Delta})^2}. \tag{2.14}$$

Здесь $M(\xi^*, j)$ — точность на `test` при seed j, $\bar{\Delta}$ — средняя разность, s_Δ — её стандартное отклонение. Приведите все три Δ_j на одном графике.

Порог приёмки. Вывод «архитектуры различаются в режиме ξ^*» установлен, если все три Δ_j одного знака и $|\bar{\Delta}| \geq 2s_\Delta$. Иначе формулировка — «различие не установлено». В отчёте нужны все три Δ_j, а не только среднее. Расхождение с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Три seeds описывают разброс начальных весов и порядка batch. Разброс из-за выбора подмножества $\mathcal{D}_r$ и рецепта обучения они не охватывают. Напишите это в разделе об ограничениях.

10 Контрфактические вопросы

Ответ опирается на ваши числа и `RUN_ID`. Если нужен ещё один запуск или оценка без обучения, выполните их.

1. Как изменилась $\Delta(r)$ от наименьшей доли профиля до $r = 1$? Можно ли по этим точкам предсказать Δ при ещё большем наборе? Какие факторы, кроме объёма данных, могут сохранить преимущество CNN?
2. Возьмите число эпох, которое дают S шагов при $r = 1$. Сколько это шагов на наименьшей доле по формуле (2.10)? Обучите CNN с этим числом шагов. Как изменилась точность относительно запуска E1 на той же доле?
3. Во сколько раз различаются $\mathcal{C}_m$ и время обучения ваших моделей? Выполнен ли режим равных параметров? А режим равных вычислений?
4. Уменьшите P вдвое. По (2.3) и (2.7) найдите новые N' и $\mathcal{C}_{\text{block}}$ и сверьте с профилировщиком. Во сколько раз выросло слагаемое $2N'^2D$? Почему число параметров почти не изменилось?
5. Сравните падение точности при сдвигах на 2, 4 и 8 пикселей. Какие из них кратны P, а какие — общему шагу CNN? Сдвиги на 2 и 4 пикселя покрыты политикой A_1, сдвиг на 8 — нет. Что лучше объясняет ваши числа: формула (2.2) или аугментация? Если `translation` нет в профиле, сделайте эту оценку отдельно. Обучение не нужно.

11 Обязательные тесты

1. Число патчей равно $N = HW/P^2$ (2.3). С классовым токеном длина последовательности равна $N + 1$. Разрезание на патчи и обратная сборка возвращают исходное изображение.
2. Обе модели дают логиты формы $(B, 10)$ и признаки формы (B, d). Профилировщик не меняет параметры модели.
3. Число операций одиночной свёртки совпадает с (2.6).
4. Подмножества воспроизводимы, вложены (2.12) и сбалансированы по классам. Запуск делает ровно S шагов при любом r.
5. Политика A_0 не содержит случайных преобразований. Два прохода по одному объекту дают один и тот же тензор.
6. Искажения не меняют метки. Уровень $s = 0$ возвращает исходное изображение. Одинаковый seed даёт одинаковое искажённое изображение.

12 Что сдать

Сдаются модули заданий 1–3, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт из шести частей. Каждое число ссылается на `RUN_ID`. Графики строятся из сохранённых файлов.

1. **Профиль и протокол:** доли, искажения, A_2, модели, S, B, скорости обучения, режим сравнения.
2. **Ресурсы:** $|\theta_m|$, $\mathcal{C}_m$, память, задержка, пропускная способность, время обучения.
3. **E1:** графики $M(r)$ и $\Delta(r)$, число эпох, r^*.
4. **E2:** графики $M_c(s)$, таблица $\delta_c(s)$ и $\overline{M}_c$, смена лидера или её отсутствие.
5. **E3 и карта:** гипотеза, все Δ_j, вывод и карта $\Delta(\xi)$ с перечнем зафиксированных факторов.
6. **Контрфактические вопросы и ограничения.**

13 Критерии завершения и оценивания

Работа завершена, если одновременно выполнено следующее.

- Эксперименты E1–E3 выполнены, у каждого результата есть `RUN_ID`, тесты проходят.
- Режим сравнения указан, и модели ему соответствуют.
- Файл `hypothesis.md` зафиксирован в Git до запусков E3.
- Карта $\Delta(\xi)$ построена, зафиксированные факторы перечислены.
- На контрфактические вопросы даны ответы со своими числами.

Веса те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Аугментация как вмешательство (Augmentation as inductive intervention).** На наименьшей доле профиля и при $r = 1$ обучите обе модели с политиками A_0 и A_2. Запуски с A_1 возьмите из E1. Выигрыш $G(A_k) = M(A_k) - M(A_0)$. Гипотеза: сильная аугментация частично заменяет ViT отсутствующие встроенные предположения [21, 19]. Отсутствие эффекта допустимо. Если A_2 ухудшила CNN и улучшила ViT, какие две разные причины нужно проверить до объяснения через индуктивные смещения?
2. **Предобучение и перенос.** Возьмите кадры лиц с разметкой эмоций. Пакет готовит преподаватель. Разбиение идёт по актёрам, единица анализа — клип, часть `cal` не читается («Протокол данных курса»). Сравните CNN и ViT близкого размера, предобученные на одном наборе, например ResNet-50 и DeiT-S [3, 10]. Режимы — линейная проба и полное дообучение [9]. В пробе учится только классификатор, а модель и буферы BatchNorm заморожены. Метрика — macro-F1. Запишите имя весов, разрешение и нормализацию. Сохраняется ли порядок из E1 [26]? О переносе — [2, разд. 15.2].

3. **Сравнение представлений.** Для моделей E1 при $r = 1$ извлеките представления трёх глубин. Обучите на них линейный классификатор по части `train` и измерьте точность на `val`. Для пар слоёв вычислите на 2000 изображениях `val` линейную CKA [27, 28]. Матрицы X и Y центрированы и построены по одним и тем же изображениям: $\mathrm{CKA}(X, Y) = \|Y^\top X\|_F^2 / (\|X^\top X\|_F \, \|Y^\top Y\|_F)$, где $\|\cdot\|_F$ — норма Фробениуса. Диаграмма t-SNE сходства не доказывает.

15 Контрольные вопросы

1. Что такое индуктивное смещение? Назовите смещения свёрточного слоя.
2. Почему общие веса уменьшают число параметров и помогают на малых данных?
3. Почему внимание без позиционных вложений не различает порядок патчей?
4. Как размер патча P влияет на число токенов и на вычисления (2.7)?
5. Чем эффективность по данным отличается от эффективности по параметрам?
6. Почему сравнение без $|\theta_m|$ и $\mathcal{C}_m$ может быть неверным?
7. Почему число эпох и число шагов дают разные кривые $M(r)$?
8. Чем устойчивость к искажениям отличается от обычного обобщения?

16 Итог

Сравнение «архитектура A лучше B» заменяется формулировкой с объёмом данных, равенством параметров и видом искажения. При смене режима преимущество может уменьшиться, исчезнуть или сменить знак. Индуктивное смещение помогает, когда оно соответствует данным и данных мало.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Documentation: torchvision.models. https://docs.pytorch.org/vision/stable/models.html

[4] PyTorch Documentation: torchvision.transforms. https://docs.pytorch.org/vision/stable/transforms.html

[5] PyTorch Documentation: torch.nn.MultiheadAttention. https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html

[6] PyTorch Documentation: torch.nn.Module (хуки `register_forward_hook`). https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

[7] PyTorch Documentation: torch.utils.benchmark. https://docs.pytorch.org/docs/stable/benchmark_utils.html

[8] Hugging Face Transformers Documentation: Vision Transformer (ViT). https://huggingface.co/docs/transformers/model_doc/vit

[9] Hugging Face Community Computer Vision Course, Unit 3: Transfer Learning and Fine-tuning Vision Transformers for Image Classification. https://huggingface.co/learn/computer-vision-course/unit3/vision-transformers/vision-transformers-for-image-classification

[10] Hugging Face timm Documentation. https://huggingface.co/docs/timm/index

[11] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. *Proc. CVPR*, pp. 770–778, 2016.

[12] A. Vaswani, N. Shazeer, N. Parmar et al. Attention is all you need. *Advances in NeurIPS*, 2017.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al. An image is worth 16x16 words: Transformers for image recognition at scale. *Proc. ICLR*, 2021.

[14] J.-B. Cordonnier, A. Loukas, M. Jaggi. On the relationship between self-attention and convolutional layers. *Proc. ICLR*, 2020.

[15] R. Zhang. Making convolutional networks shift-invariant again. *Proc. ICML*, PMLR 97:7324–7334, 2019.

[16] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[17] D. Hendrycks, T. Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. *Proc. ICLR*, 2019.

[18] S. Bhojanapalli, A. Chakrabarti, D. Glasner et al. Understanding robustness of transformers for image classification. *Proc. ICCV*, pp. 10231–10241, 2021.

[19] A. Steiner, A. Kolesnikov, X. Zhai et al. How to train your ViT? Data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research*, 2022.

[20] R. Wightman, H. Touvron, H. Jégou. ResNet strikes back: An improved training procedure in timm. arXiv:2110.00476, 2021.

[21] H. Touvron, M. Cord, M. Douze et al. Training data-efficient image transformers & distillation through attention. *Proc. ICML*, PMLR 139:10347–10357, 2021.

[22] E. D. Cubuk, B. Zoph, J. Shlens, Q. V. Le. RandAugment: Practical automated data augmentation with a reduced search space. *Advances in NeurIPS*, 2020.

[23] S. G. Müller, F. Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. *Proc. ICCV*, pp. 774–782, 2021.

[24] H. Zhang, M. Cissé, Y. N. Dauphin, D. Lopez-Paz. mixup: Beyond empirical risk minimization. *Proc. ICLR*, 2018.

[25] S. Yun, D. Han, S. J. Oh et al. CutMix: Regularization strategy to train strong classifiers with localizable features. *Proc. ICCV*, pp. 6023–6032, 2019.

[26] S. Kornblith, J. Shlens, Q. V. Le. Do better ImageNet models transfer better? *Proc. CVPR*, pp. 2661–2671, 2019.

[27] S. Kornblith, M. Norouzi, H. Lee, G. Hinton. Similarity of neural network representations revisited. *Proc. ICML*, PMLR 97:3519–3529, 2019.

[28] M. Raghu, T. Unterthiner, S. Kornblith et al. Do vision transformers see like convolutional neural networks? *Advances in NeurIPS*, 2021.